

Handling Imbalanced Data in Machine Learning

Hazem Mohamed
AI Engineer

1. Handling Imbalanced Data in Machine Learning

1. Handling Imbalanced Data in Machine Learning

- Class imbalance is one of the most common problems in classification tasks.

It happens when:

- one class has many samples
- another class has very few samples
- Example:
 - 95% Non-Fraud
 - 5% Fraud
- In this situation, machine learning models often become biased toward the majority class.

2. Why Imbalanced Data is Dangerous

2. Why Imbalanced Data is Dangerous

- Imagine:
 - 950 normal transactions
 - 50 fraud transactions
- A model can predict: Everything is normal
- and still achieve: [Accuracy = 95%]
- The model appears strong but completely fails to detect fraud cases.
- This is why:
 - Accuracy becomes misleading
 - Recall, Precision, F1-score, and ROC-AUC become more important

3. Main Approaches for Handling Imbalance

3. Main Approaches for Handling Imbalance

- There are three common approaches:
 1. Class Weights
 2. SMOTE (Oversampling)
 3. Undersampling
- Each method works differently and should be used in different situations.

4. Class Weights

4. Class Weights

- What Does It Do?

Class weights increase the importance of the minority class during training.

- The model gets penalized more when:
 - minority samples are classified incorrectly
- This forces the algorithm to pay more attention to the minority class.

4. Class Weights

- How Does It Work?

The algorithm automatically assigns:

- larger weight to minority class
- smaller weight to majority class

- Example:

Class	Weight
Minority Class	1
Minority Class	5

- Errors on the minority class become more expensive.

4. Class Weights

- **Example in Scikit-Learn**

```
LogisticRegression(class_weight='balanced')
```

4. Class Weights

- Advantages
 1. Simple to apply
 2. No data duplication
 3. No synthetic samples generated
 4. Works well with many linear models and tree-based models
 5. Reduces bias toward majority class
- Disadvantages
 1. Sometimes not enough for severe imbalance
 2. Can increase false positives
 3. Some models respond better than others

4. Class Weights

- When to Use Class Weights
 1. Medium imbalance
 2. Small datasets
 3. Logistic Regression
 4. SVM
 5. Tree-based models
- When NOT to Use Class Weights
 1. Extremely imbalanced datasets
 2. When recall is still very poor after weighting

5. SMOTE (Synthetic Minority Oversampling Technique)

5. SMOTE

(Synthetic Minority Oversampling Technique)

- What Does It Do?
 - SMOTE creates NEW synthetic minority samples instead of simply duplicating existing ones.
 - It increases the size of the minority class.

- How Does SMOTE Work?

SMOTE:

- Selects a minority sample
- Finds nearest minority neighbors
- Generates synthetic samples between them

Instead of copying data:

- it creates artificial but realistic samples

5. SMOTE

(Synthetic Minority Oversampling Technique)

- **Example**

Original minority samples:

- A ----- B
- SMOTE generates new synthetic points between A and B.

- **Example in Python**

```
from imblearn.over_sampling import SMOTE
smote = SMOTE()
X_train_smote, y_train_smote =
smote.fit_resample(X_train, y_train)
```


5. SMOTE

(Synthetic Minority Oversampling Technique)

- **Advantages**

- Improves minority class learning
- Often increases Recall
- Better than simple duplication
- Useful for severe imbalance

- **Disadvantages**

- Can create noisy synthetic samples
- May increase overfitting
- Can generate unrealistic data
- Sometimes increases false positives

5. SMOTE

(Synthetic Minority Oversampling Technique)

- **Important Rule**

1. SMOTE should ONLY be applied to:
 - training data
2. Never apply SMOTE before train-test split.
3. Correct workflow:
 1. Split data
 2. Apply preprocessing on training set
 3. Apply SMOTE ONLY on training data
 4. Train model
 5. Evaluate on untouched test data

5. SMOTE

(Synthetic Minority Oversampling Technique)

- When to Use SMOTE
 - Severe imbalance
 - Need higher Recall
 - Sufficient dataset size
 - Minority class is important
- Examples:
 - Fraud detection
 - Disease prediction
 - Customer churn prediction

5. SMOTE

(Synthetic Minority Oversampling Technique)

- When NOT to Use SMOTE
 - Very small datasets
 - Noisy datasets
 - High risk of overfitting
 - When synthetic samples may be unrealistic

6. Undersampling

6. Undersampling

What Does It Do?

- Undersampling reduces the number of majority class samples.
- Instead of increasing minority samples:
 - it removes majority samples

How Does It Work?

- Example:
- Before: 10,000 majority, 500 minority
- After undersampling: 500 majority, 500 minority
- The dataset becomes balanced by removing data.

6. Undersampling

Example in Python

```
from imblearn.under_sampling import  
RandomUnderSampler
```

```
rus = RandomUnderSampler()
```

```
X_resampled, y_resampled =  
rus.fit_resample(X_train, y_train)
```

6. Undersampling

- **Advantages**
 - Fast
 - Reduces training time
 - Simple to apply
 - Useful for very large datasets
- **Disadvantages**
 - Removes potentially useful information
 - Can hurt model performance
 - May reduce generalization ability

6. Undersampling

- **When to Use Undersampling**
 - Very large datasets
 - Majority class is extremely dominant
 - Training speed matters
- **When NOT to Use Undersampling**
 - Small datasets
 - When losing information is dangerous

7. Comparing the Methods

7. Comparing the Methods

Method	Main Idea	Best For	Main Risk
Class Weights	Penalize minority mistakes	Medium imbalance	More false positives
SMOTE	Generate synthetic minority samples	Severe imbalance	Overfitting
Undersampling	Remove majority samples	Huge datasets	Information loss

8. Which Method Should You Try First?

8. Which Method Should You Try First?

- Recommended order:
- **Step 1**
 - Try baseline model WITHOUT imbalance handling.
- **Step 2**
 - Try: `class_weight='balanced'`
 - Simple and effective.

8. Which Method Should You Try First?

- **Step 3**
 - If Recall is still weak:
 - try SMOTE
- **Step 4**
 - Use undersampling only when:
 - dataset is very large

9. Important Practical Notes

9. Important Practical Notes

Imbalance Handling Changes Metrics

- After applying imbalance methods:
 - Recall usually increases
 - Precision may decrease
 - False positives may increase
- This is normal.

9. Important Practical Notes

Do NOT Judge Using Accuracy Alone

- After imbalance handling:
 - Accuracy may decrease
 - BUT the model may become much better
- Always inspect:
 - Precision
 - Recall
 - F1-score
 - ROC-AUC
 - Confusion Matrix

9. Important Practical Notes

Common Beginner Mistake

- Applying SMOTE before train-test split.

This causes:

- data leakage
 - unrealistic performance
 - misleading evaluation
- Always:
 - split first
 - resample training data only

10. Key Takeaways

10. Key Takeaways

- **Imbalanced datasets require special handling.**
 - Accuracy alone is not reliable.
 - Class weights adjust learning importance.
 - SMOTE creates synthetic minority samples.
 - Undersampling removes majority samples.
 - Choose the method based on:
 - dataset size
 - imbalance severity
 - business goal

10. Key Takeaways

- Always evaluate using:
 - Recall
 - Precision
 - F1-score
 - ROC-AUC
 - Confusion Matrix

Follow to Understand

AI Engineer | Machine Learning

- From theory → to real implementation

Follow for practical ML insights that actually make sense